Confidencial – Propiedad de VirtualSoft.
Prohibida su reproducción o distribución sin autorización.

Integración LOTOBOLA

Describe la configuración y el uso de la API del proveedor, incluyendo las operaciones de autenticación, apuestas, ganancias, devoluciones y consulta de saldo. Esta sección define la estructura de las peticiones y respuestas, los parámetros requeridos para cada operación y el mecanismo de autenticación mediante firmas HMAC, incluyendo la construcción del contenido a firmar y la generación de la firma requerida para cada petición.

VIRTUALSOFT
21/09/2026

Versión: 1.0

Tabla de contenido

1. Propósito 2
2. Alcance 2
3. Requisitos previos 2
4. Autenticación 2
5. Métodos transaccionales 2
   - 5.1. session/validate 3
     - 5.1.1. Parámetros 3
     - 5.1.2. Respuesta 3
   - 5.2. wallet/reserve 4
     - 5.2.1. Parámetros 4
     - 5.2.2. Respuesta 5
   - 5.3. wallet/payout 5
     - 5.3.1. Parámetros 6
     - 5.3.2. Respuesta 6
   - 5.4. wallet/void 7
     - 5.4.1. Parámetros 7
     - 5.4.2. Respuesta 8
   - 5.5. wallet/commit 8
     - 5.5.1. Parámetros 8
     - 5.5.2. Respuesta 9
6. Códigos de respuesta en caso de error 9
7. Control de versiones 10

1. Propósito

Proporcionar una guía técnica para la integración con la API del proveedor, definiendo los lineamientos necesarios para configurar, autenticar y consumir correctamente los servicios disponibles.

2. Alcance

Esta documentación cubre los aspectos técnicos necesarios para la integración con la API del proveedor, desde la configuración inicial hasta el consumo y autenticación de las operaciones disponibles.

3. Requisitos previos

Antes de iniciar la integración, se deben cumplir los siguientes requisitos:

- **IP autorizada:** Registrar las direcciones IP de **VIRTUALSOFT** en lista blanca
- **Secret Key:** Disponer de la clave secreta requerida para la generación de firmas HMAC.
- **API Endpoint:** Contar con el endpoint correspondiente al ambiente donde se realizará la integración.
- **Firma HMAC:** Implementar la generación y el envío de la firma en el header correspondiente de cada petición, de acuerdo con el mecanismo de autenticación definida por **VIRTUALSOFT**.

4. Autenticación

Todas las solicitudes deben estar firmadas con el algoritmo **HMAC-SHA256** y debe ser enviado en los **headers** con el nombre **X-REQUEST-SIGNATURE**

El hash se construye con la data sin formatear (*RAW*) y agregando el **SECRET-KEY** compartida por nosotros

5. Métodos transaccionales

Tener en cuenta que en los ejemplos se formateará el JSON para facilitar la legibilidad, para el hash debe usarse un JSON sin formato ni espacios

Método	URL
POST	https://apidevintegrations.virtualsoft.tech/integrations/casino/genericTransactional/api/lotobola/{metodo}

5.1. session/validate

Método utilizado para **obtener la información del usuario que inicia el juego**, permitiendo validar su identidad y disponer de los datos necesarios para establecer correctamente la sesión de juego.

5.1.1. Parámetros

Campo	Tipo	Obligatorio	Descripción
token	string	si	Token de la sesión del usuario

Ejemplo:

```
{
    "token": "183P12941P86yscs5pvltmkeik2kgd"
}
```

5.1.2. Respuesta

Campo	Tipo	Obligatorio	Descripción
success	boolean	si	Indica si la respuesta es de éxito o de error
data	object	si	Objecto que contiene la información del usuario
data.balance	int	si	Saldo actual del usuario
data.playerId	string	si	Id del usuario que abre el juego
data.country	string	si	Siga del país al que pertenece el usuario que abre el juego
data.language	string	si	Sigla del idioma al que pertenece el usuario que abre el juego
data.currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "success": true,
    "data": {
        "balance": 3000,
```

```
        "playerId": "12941",
        "country": "CO",
        "language": "ES",
        "currency": "COP"
    }
}
```

5.2. wallet/reserve

Método utilizado para realizar apuestas mediante un movimiento de billetera. Al ejecutar la operación, se descuenta del saldo del usuario el monto correspondiente a la apuesta definido en el campo **amount** de la transacción.

5.2.1. Parámetros

Campo	Tipo	Obligatorio	Descripción
token	string	si	Token de la sesión del usuario
playerId	string	si	Id del usuario que abre el juego
gameId	string	si	Id del juego que abrirá el usuario
transactionId	string	si	Id único de la transacción
amount	int	si	Valor por debitar de la billetera del usuario representado en centavos
roundId	string	si	Identificador que vincula una operación de apuesta con una operación de ganancia
currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "gameId": "lotobola123",
    "transactionId": "TX-1788276260672a1",
    "amount": 1000,
    "roundId": "5fbf16b6-c592-4549-adeb-7e70be84e022",
    "currency": "COP",
    "playerId": "12941",
    "token": "183P12941P86yscs5pvltmkeik2kgd"
}
```

5.2.2. Respuesta

Campo	Tipo	Obligatorio	Descripción
success	boolean	si	Indica si la respuesta es de éxito o de error
data	object	si	Objecto que contiene la información del usuario
data.balance	int	si	Saldo actual del usuario
data.playerId	string	si	Id del usuario que abre el juego
data.currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "success": true,
    "data": {
        "balance": 3000,
        "playerId": "12941",
        "currency": "COP"
    }
}
```

5.3. wallet/payout

Método utilizado para otorgar ganancias al usuario mediante un movimiento de billetera, acreditando en su saldo el monto indicado en el campo **amount**.

El método también puede ser invocado con **amount = 0** en los siguientes casos:

- Cuando se haya realizado una operación de tipo **reserve** y sea necesario completar el flujo de la transacción.
- Para cerrar una ronda de juego, incluso cuando el usuario no haya obtenido ninguna ganancia.

En estos casos, aunque no se realice una acreditación de saldo, la operación permite completar correctamente el ciclo de la transacción o cerrar la ronda correspondiente.

5.3.1. Parámetros

Campo	Tipo	Obligatorio	Descripción
playerId	string	si	Id del usuario que abre el juego
gameId	string	si	Id del juego que abrirá el usuario
transactionId	string	si	Id único de la transacción
amount	int	si	Valor por debitar de la billetera del usuario representado en centavos
roundId	string	si	Identificador que vincula una operación de apuesta con una operación de ganancia
currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego
closeRound	boolean	no	Indicador para cerrar la ronda de una transacción, sino se envía por defecto es `false`

Ejemplo:

```
{
    "gameId": "lotobola123",
    "transactionId": "TX-1788276260672a2",
    "amount": 1000,
    "roundId": "5fbf16b6-c592-4549-adeb-7e70be84e022",
    "currency": "COP",
    "closeRound":false,
    "playerId": "12941"
}
```

5.3.2. Respuesta

Campo	Tipo	Obligatorio	Descripción
success	boolean	si	Indica si la respuesta es de éxito o de error
data	object	si	Objecto que contiene la información del usuario
data.balance	int	si	Saldo actual del usuario
data.playerId	string	si	Id del usuario que abre el juego

data.currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "success": true,
    "data": {
        "balance": 3000,
        "playerId": "12941",
        "currency": "COP"
    }
}
```

5.4. wallet/void

Método utilizado para realizar el **rollback** de una transacción, devolviendo a la billetera del usuario el valor correspondiente a la apuesta realizada en dicha transacción. Al ejecutar el **rollback**, se acredita nuevamente en la billetera el monto definido en el campo **amount** de la transacción asociada.

5.4.1. Parámetros

Campo	Tipo	Obligatorio	Descripción
playerId	string	si	Id del usuario que abre el juego
gameId	string	si	Id del juego que abrirá el usuario
transactionId	string	si	Id de la transacción a la que se le hará rollback
amount	int	no	Valor que debe ser devuelto al usuario

Ejemplo:

```
{
    "gameId": "lotobola123",
    "transactionId": "TX-1788276260672a1",
    "amount": 1000,
    "playerId": "12941"
}
```

5.4.2. Respuesta

Campo	Tipo	Obligatorio	Descripción
success	boolean	si	Indica si la respuesta es de éxito o de error
data	object	si	Objecto que contiene la información del usuario
data.balance	int	si	Saldo actual del usuario
data.playerId	string	si	Id del usuario que abre el juego
data.currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "success": true,
    "data": {
        "balance": 3000,
        "playerId": "12941",
        "currency": "COP"
    }
}
```

5.5. wallet/commit

Método utilizado para consultar el saldo actual del usuario en su billetera. También puede utilizarse para verificar la vigencia de la sesión, validando que el usuario continúe activo y pueda seguir interactuando con el juego.

5.5.1. Parámetros

Campo	Tipo	Obligatorio	Descripción
token	string	si	Token de la sesión del usuario
gameId	string	si	Id del juego que abrirá el usuario
currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
```

```
    "gameId": "lotobola123",
    "currency": "COP",
    "token": "183P12941P86yscs5pvltmkeik2kgd"
}
```

5.5.2. Respuesta

Campo	Tipo	Obligatorio	Descripción
success	boolean	si	Indica si la respuesta es de éxito o de error
data	object	si	Objecto que contiene la información del usuario
data.balance	int	si	Saldo actual del usuario
data.playerId	string	si	Id del usuario que abre el juego
data.currency	string	si	Sigla de la moneda al que pertenece el usuario que abre el juego

Ejemplo:

```
{
    "success": true,
    "data": {
        "balance": 3000,
        "playerId": "12941",
        "currency": "COP"
    }
}
```

6. Códigos de respuesta en caso de error

El código de error 201 es manejado como idepotence

Código	Mensaje
401	El usuario cuenta con autoexclusión
403	Fondos insuficientes
404	Usuario no existe
500	Sitio en mantenimiento

403	Token de usuario inactivo o expirado
404	Juego no disponible
401	Usuario tiene contingencia
401	El valor de apuesta no puede ser negativo
401	Juego inactivado
201	Transacción ya procesada
401	La ronda ya fue cerrada
404	Limite de apuesta fue alcanzado
201	Transacción no existe
401	Moneda invalida
401	La firma no coincide o no fue enviada

7. Control de versiones

Versión	Fecha	Autor	Descripción
1.0	21/09/2026	Juan Esteban López Arévalo	Versión inicial